CHAPTER 6

Integrity and Security

Integrity constraints ensure that changes made to the database by authorized users do not result in a loss of data consistency. Thus, integrity constraints guard against accidental damage to the database.

We have already seen two forms of integrity constraints for the E-R model in Chapter 2:

- **Key declarations**—the stipulation that certain attributes form a candidate key for a given entity set.
- **Form of a relationship**—many to many, one to many, one to one.

In general, an integrity constraint can be an arbitrary predicate pertaining to the database. However, arbitrary predicates may be costly to test. Thus, we concentrate on integrity constraints that can be tested with minimal overhead. We study some such forms of integrity constraints in Sections 6.1 and 6.2, and cover a more complex form in Section 6.3. In Chapter 7 we study another form of integrity constraint, called "functional dependency," which is primarily used in the process of schema design.

In Section 6.4 we study *triggers*, which are statements that are executed automatically by the system as a side effect of a modification to the database. Triggers are used to ensure some types of integrity.

In addition to protecting against accidental introduction of inconsistency, the data stored in the database need to be protected from unauthorized access and malicious destruction or alteration. In Sections 6.5 through 6.7, we examine ways in which data may be misused or intentionally made inconsistent, and present security mechanisms to guard against such occurrences.

6.1 Domain Constraints

We have seen that a domain of possible values must be associated with every attribute. In Chapter 4, we saw a number of standard domain types, such as integer

types, character types, and date/time types defined in SQL. Declaring an attribute to be of a particular domain acts as a constraint on the values that it can take. Domain constraints are the most elementary form of integrity constraint. They are tested easily by the system whenever a new data item is entered into the database.

It is possible for several attributes to have the same domain. For example, the attributes *customer-name* and *employee-name* might have the same domain: the set of all person names. However, the domains of *balance* and *branch-name* certainly ought to be distinct. It is perhaps less clear whether *customer-name* and *branch-name* should have the same domain. At the implementation level, both customer names and branch names are character strings. However, we would normally not consider the query "Find all customers who have the same name as a branch" to be a meaningful query. Thus, if we view the database at the conceptual, rather than the physical, level, *customer-name* and *branch-name* should have distinct domains.

From the above discussion, we can see that a proper definition of domain constraints not only allows us to test values inserted in the database, but also permits us to test queries to ensure that the comparisons made make sense. The principle behind attribute domains is similar to that behind typing of variables in programming languages. Strongly typed programming languages allow the compiler to check the program in greater detail.

The **create domain** clause can be used to define new domains. For example, the statements:

create domain *Dollars* **numeric**(12,2)
create domain *Pounds* **numeric**(12,2)

define the domains *Dollars* and *Pounds* to be decimal numbers with a total of 12 digits, two of which are placed after the decimal point. An attempt to assign a value of type *Dollars* to a variable of type *Pounds* would result in a syntax error, although both are of the same numeric type. Such an assignment is likely to be due to a programmer error, where the programmer forgot about the differences in currency. Declaring different domains for different currencies helps catch such errors.

Values of one domain can be *cast* (that is, converted) to another domain. If the attribute A or relation r is of type *Dollars*, we can convert it to *Pounds* by writing

cast $r.A$ **as** *Pounds*

In a real application we would of course multiply $r.A$ by a currency conversion factor before casting it to pounds. SQL also provides **drop domain** and **alter domain** clauses to drop or modify domains that have been created earlier.

The **check** clause in SQL permits domains to be restricted in powerful ways that most programming language type systems do not permit. Specifically, the **check** clause permits the schema designer to specify a predicate that must be satisfied by any value assigned to a variable whose type is the domain. For instance, a **check** clause can ensure that an hourly wage domain allows only values greater than a specified value (such as the minimum wage):